Negotiate, Then Solve

Solver-Grounded Negotiation for Conflict-Aware University Timetabling

Final Report

Team Farali Chevdo

Shubhadeep Mandal	2025201056	Achyutananda Sahoo	2025202028
Satyajit Priyadarshi	2025201055	Rakesh C	2026901015

Code: https://github.com/Seronic2001/negotiate-then-solve

Abstract. University timetables change every week after they are published, through requests written in plain language that often conflict. We study *solver-grounded negotiation*: when a request cannot be met, minimal unsatisfiable subsets (MUS) locate the conflict, minimal correction sets (MCS) define every possible concession, CP-SAT re-solves each into a concrete timetable, and a language model may offer people *only* these verified options, ranked by priority tiers and a concession ledger. We build the whole system, from a plain-language message to a human-approved timetable, as a web application in which small fine-tuned models (Qwen3.5-4B with one LoRA adapter per task) only *read* language into typed vocabularies and code does every computation. On 60 negotiation scenarios with planted conflicts (180 paired runs), solver-grounded negotiation reaches the oracle's outcome in **89.4%** of runs, against 75.6% for solver-filtered model inventions ($p = 4.6 \times 10^{-6}$) and 83.3% for unverified ones ($p = 0.013$), with an oracle gap of 0.04 against 0.63 and 0.44 and no invalid option offered. The fine-tuned readers beat a much larger prompted model on reading requests (outcome match 96.4–98.1%), policy checks and stakeholder replies (100% tool accuracy); a lookup vocabulary lifts long, mixed chat messages from 111 to 140 of 200; and conversational memory lifts follow-ups from 146 to 388 of 400. Ablations show that grounded explanations remove private-reason leaks (0 against 12), that the concession ledger does not yet spread burden more evenly (H4 not supported), and that most of the remaining errors come from training data gaps, not from the architecture.

1 Introduction

A university timetable is built once a semester and then changes every week. Teachers go to conferences, labs close for maintenance, guest lecturers come with fixed windows, and students report clashes. These requests arrive as free text through a portal, email or messaging, and an academic office resolves them by hand. One change often silently breaks another constraint, and when two requests cannot both be met there is no principled way to decide *who should give way*, so decisions look arbitrary to the people affected.

Language models can read such requests, and constraint solvers can decide whether a timetable is valid, but neither alone solves the problem. A model that writes a timetable breaks hard constraints; a solver alone imposes the cheapest repair without asking anyone, though the people involved often have flexibility it cannot see. The common combination, in which a model proposes options and a solver filters them, still leaves the option space to the model's imagination.

Research question

Can solver-derived conflict sets constrain an LLM negotiator so that it resolves infeasible timetable requests with fewer negotiation rounds and lower disruption than LLM-invented options, even when those options are filtered by the same solver before they are offered?

Hypotheses. **H1** invented options cost rounds or regeneration calls; **H2** ours resolves at least as often as the filtered baseline with fewer rounds, less disruption and a smaller oracle gap; **H3** grounded explanations are more faithful and leak no private reasons; **H4** a concession ledger spreads the burden of giving way more evenly than flat weights.

Contributions.

1. **Solver-grounded negotiation**: MCS enumeration defines the space of offers, measured against generate-and-filter on a benchmark with planted conflicts and an oracle (Sections 4.4, 6.1).

2. **A complete, role-aware system**: portal, email and messaging intake; a request pipeline with seven layers from reading to human approval; a chat that answers timetable and rule questions and takes requests in the same message; and a semester planner. It runs end to end, and offline.
3. **Read with a model, compute with code**: four small LoRA adapters on one 4B base, each reading language into a typed vocabulary (request intents, a lookup vocabulary, follow-up rewrites, tool calls), with every name, date, slot, rule and timetable fact resolved and checked by code (Section 4.2).
4. **A tool-calling coordinator agent** as an alternative architecture, trained with SFT and DPO on its own verified trajectories (Section 4.6).
5. **Evaluation and ablations** for every capability, each with a baseline (Sections 6 and 7).

2 Literature Review

Automated timetabling. University course timetabling is a long-studied constraint problem [1], and constraint programming builds and repairs timetables well; CP-SAT [6] is a strong general solver for it. These tools need fully formal constraints: they cannot read a request, explain why it cannot be met in words a teacher accepts, or negotiate who should give way.

Explaining infeasibility. When constraints conflict, a minimal unsatisfiable subset (MUS) is a smallest set that cannot hold together, and a minimal correction set (MCS) is a smallest set whose removal restores feasibility [5]. MCSs are the hitting-set duals of MUSs, so enumerating them lists every minimal repair. This work has served a single modeller fixing a model; we use MCSs as the space of offers when negotiating with many stakeholders.

Language models with solvers. Language models plan poorly on their own but help when an external verifier checks what they propose [3]. In such systems, and in scheduling assistants built the same way, the model proposes and the solver verifies. We reverse the roles for negotiation: the solver proposes every option, and the model only explains the options and reads the replies.

Negotiation and tool-using agents. Dialogue agents that learn to negotiate, and language-model stakeholders negotiating in multi-agent games, make free-form offers with no model of what is feasible. Agents that call tools in a loop are the basis of our coordinator agent; the pipeline uses tool calls only to read stakeholder replies, one typed call per reply.

Reading language into structure. Parsing language into a small typed target works best when generation is held to the target's grammar; every adapter here is held to a JSON schema, and code resolves names and dates rather than the model. Rewriting a question so it stands without its conversation is the task of our memory adapter.

Retrieval. Retrieval-augmented generation grounds a model's answers in documents [4]. Our policy check retrieves handbook rules by fusing word matching (BM25) and dense embeddings, and may cite only the rules it was shown.

Fine-tuning small models. LoRA trains low-rank updates on a frozen model [2], so a 4B model can be adapted on free GPUs and several adapters can share one base. Adapting every linear layer is the usual practice, continued training forgets earlier skills unless old data is replayed, and preference optimisation trains on pairs of better and worse outputs; all three shape our training (Section 4.7).

Safety and explanation. Instructions hidden in the text a model reads can take over model-integrated applications; our architecture treats every message as data and lets no model change state directly. Good explanations are contrastive and selective, and must not reveal private reasons.

The gap. Prior work supplies the pieces: solvers that check, MUS/MCS that diagnose, models that read and negotiate. Missing is an interpretation layer that connects formal infeasibility analysis to multi-stakeholder negotiation, deciding which concessions are possible, whom to ask and how to explain it, without leaving the feasible space.

3 Datasets

All data is synthetic or written by the team, with the right answer known by construction: each example is generated from its answer, or checked by reading it back. Test data is always built from templates and wordings that training never sees, and timetable questions are also tested in departments the models never saw.

Dataset	Construction	Size
Department	Seeded synthetic CSE department; institute rules (lunch, at most 3 hours in a row) built in	30 teachers, 10 sections, 20 rooms, 202 classes/week, 16 weeks
A. Negotiation	Planted conflicts of 7 kinds (contention, squeeze, capacity, tradeoff, policy, deadlock, substitute); hidden stakeholder flexibility; 4 stakeholder styles; solver oracle	60 scenarios + 30 hard
B. Request corpus	Requests from known targets: clean, ambiguous, injection, rule-breaking, unauthorised; Gemini rewordings	600 + 600
B. Intent data	Recipe-built from the corpus and generators (one per kind of request); 4 quality gates; frozen test split	4,623 / 373 / 417 (v13)
B. Held-out suites	Wordings training never uses, one suite per skill; noisy and typo versions of the test split	26 suites, ≈40 each
C. Policy	Handbook (16 rules) and 4 documents (PDF, scan, photo, HTML); retrieval queries with gold rule IDs	32 rules; 106 + 40 queries
D. Replies, safety	Stakeholder replies with the one correct tool call; authority, injection, privacy and spoofing cases	100; 48
E. Multi-task	Policy, reply and explanation rows built with the run-time prompts; the model's own mistakes mined	6,570 / 561
F. Timetable questions	Templates filled from departments; long messages of 2–5 parts; Gemini rewordings kept only if they read back the same	2,354–2,976 train rows/version; 362 + 86 + 69 test questions; 200 long messages
G. Follow-ups	Conversations with the app's real answers; 16 kinds of follow-up; test-only wordings	2,520 train; 400 test; 64 plural
Agent	50 request families built from the department's own names; verified trajectories; outcome preference pairs	10 episodes per family over two departments

Table 1: Datasets.

Test sets for timetable questions. The same 362 questions (142 templates, 3 fills each) are asked in four settings: **A** the app's department; **B2, B3** departments never seen; **C** the app's department with names, the head of department and course titles shuffled, so memorised names fail; **D** a new department with shared and co-taught courses.

4 Methodology

4.1 Architecture

The system follows seven layers (Figure 1): channels (L0), a central store (L1), reading (L2), a policy check (L3), solving (L4), interpretation and negotiation (L5) and human approval (L6). A chat beside each person's timetable answers questions and takes requests. One rule runs through every part: **models read language; code computes**. No model sees the timetable, writes a timetable, or decides who may do what.

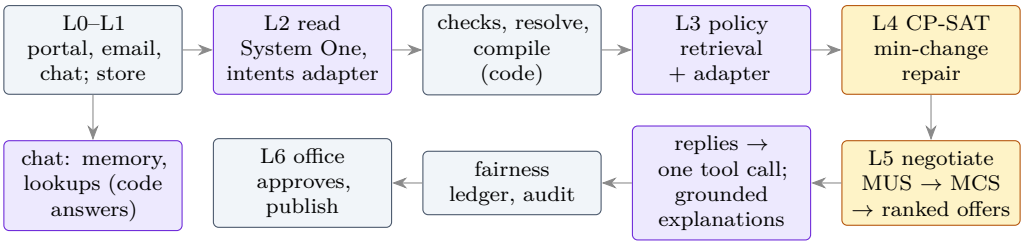


Figure 1: The request pipeline. Violet: a fine-tuned model reads or writes language; grey: deterministic code; amber: the constraint solver. Requests made with clicks skip reading entirely.

A request moves through fixed states (received, classified, policy-checked, compiled, solved, negotiating, fairness-audited, awaiting approval, published; or answered, forwarded, refused, denied, escalated, clarification requested, withdrawn); an illegal transition raises an error. Every model-backed component has a deterministic stand-in (a rule parser, a rule-based policy check, a regex question reader, template explanations), so the system keeps working, more weakly, when the model server is away; requests that would change the timetable wait up to ten minutes for the model rather than be read by the stand-in.

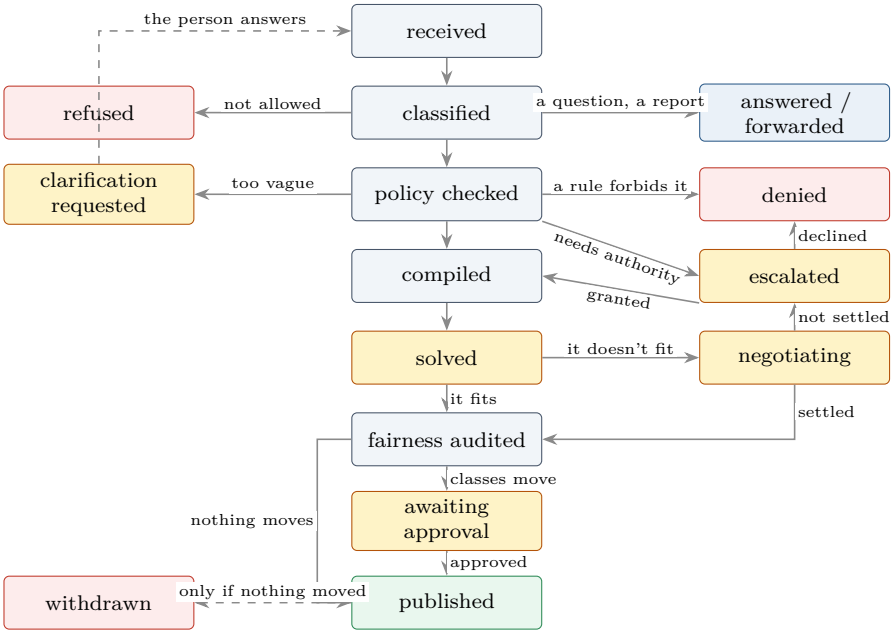


Figure 2: A request's life. Only these transitions are allowed. Amber: waiting on a person or the solver; green: in force; red: it stops; blue: answered or passed on.

4.2 Reading requests (L2)

A message is first prepared: lines that instruct the system are removed before any reader sees them (a message that is nothing else is refused unread), and shorthand is spelled out ("pls shift my DBMS lec" becomes "please shift my Database Systems lecture"). **System One**, a calibrated linear SVM over TF-IDF features, routes it. While the model server is up, the fine-tuned **intents adapter** reads every request into an `IntentOutput`: an action (compile, clarify, investigate, answer, out of scope, withdraw) and intents in 17 kinds (unavailable, avoid, only, move, swap, extra class, cancel, room closed, needs room, load limit, limit, relate, any-of, ranked, fallback, club booking, clash report). The model writes clock times ("after 14:00"), periods (`next_week`) and class descriptions ("my Monday Data Structures lecture"), never slot numbers or IDs. Code then checks the reading against the message and the timetable (repairing near misses, turning a hedged requirement into a wish and a contradiction into a question), resolves names, times, weeks and classes, assigns **priority tiers** by rule (0 physical, 1 policy, 2 commitment, 3 verified unavailability, 4 operational, 5 preference), checks authority, and compiles constraints. Any part of a message the reading did not use is reported back to the sender.

Worked example: from a message to a constraint

Message (a teacher, through the portal): *"pls shift my DBMS lec frm monday to thursday afternoon if possible"*
1. Prepared. Shorthand spelled out: "please shift my Database Systems lecture from Monday to Thursday afternoon if possible".
2. Read by the intents adapter (the model's whole output):

```
{"action": "compile", "intents": [{"kind": "move",
  "which": {"course": "Database Systems", "kind": "lecture", "day": "Mon"},
  "to": {"days": ["Thu"], "part": "afternoon"}, "hard": false}]}
```

3. Checked and resolved by code. "Database Systems" is found in the department's courses; "my … lecture on Monday" is matched to the one session the teacher holds on Monday in the published timetable; "afternoon" becomes the slots from 14:00 to 16:00 on the calendar; no period given means every week; "if possible" agrees with `hard: false`, so it stays a wish.
4. Compiled. A preference (Tier 5) that this session be placed on Thursday afternoon, owned by the teacher, with authority checked (it is their own class). The policy check then finds no rule against it, and the solver repairs the week.

4.3 Policy check (L3)

The top five rules are retrieved by hybrid search (BM25 and bge-small embeddings, fused by reciprocal rank) over the handbook and policy documents in any form (PDF, OCR of scans and photos, HTML). The policy adapter returns a verdict (allowed, needs approval, forbidden), cited rules, obligations such as a make-up class, and an alternative. Code then drops any citation of a rule it was not shown, and **nothing is denied without the rule it breaks**: an uncited denial becomes "needs approval". The examination regulations are applied by code.

4.4 Solving and negotiation (L4–L5)

Solver. CP-SAT places every class once with no clash and a fitting room (Tier 0, always enforced). Every other firm constraint C_k is guarded by an assumption literal ($a_k \Rightarrow C_k$), so conflict analysis can switch constraints off; wishes and, in repair mode, moved classes form the objective, solved tier by tier. A repair may be restricted to the neighbourhood a conflict touches.

Priority. Within a tier each constraint has a published score $\pi_k = w_1\,auth + w_2\,impact + w_3\,just + w_4\,lead + w_5\,credit - w_6\,disrupt$, and a correction option M costs $\text{cost}(M) = \sum_{k\in M}\pi_k + \lambda_1\,\text{disruption}(M) + \lambda_2\,\Delta\text{Gini}(M)$, where ΔGini is the change in how unevenly the concession burden falls.

The resolution ladder. (1) *Auto-substitute*: if the firm constraints fit together, repair and notify. (2) *Auto-relax*: wishes the repair cannot keep are dropped and their owners told. (3) *Negotiate*: find the MUS, enumerate MCSs within the relaxable tiers, each with a solver-found witness timetable, rank them by cost, and ask the owner of the cheapest with two or three verified options and a grounded explanation; a declined option rules out its day and part of day for the next offers, and a counter-offer is passed on only if CP-SAT verifies it. (4) *Escalate* on deadlock, no reply, the round limit, or a conflict only a Tier 1–2 authority can relax. Each reply becomes **exactly one typed tool call** (accept, apply reply, counter-propose, ask clarification, decline, escalate); the ladder, not the model, decides what it does.

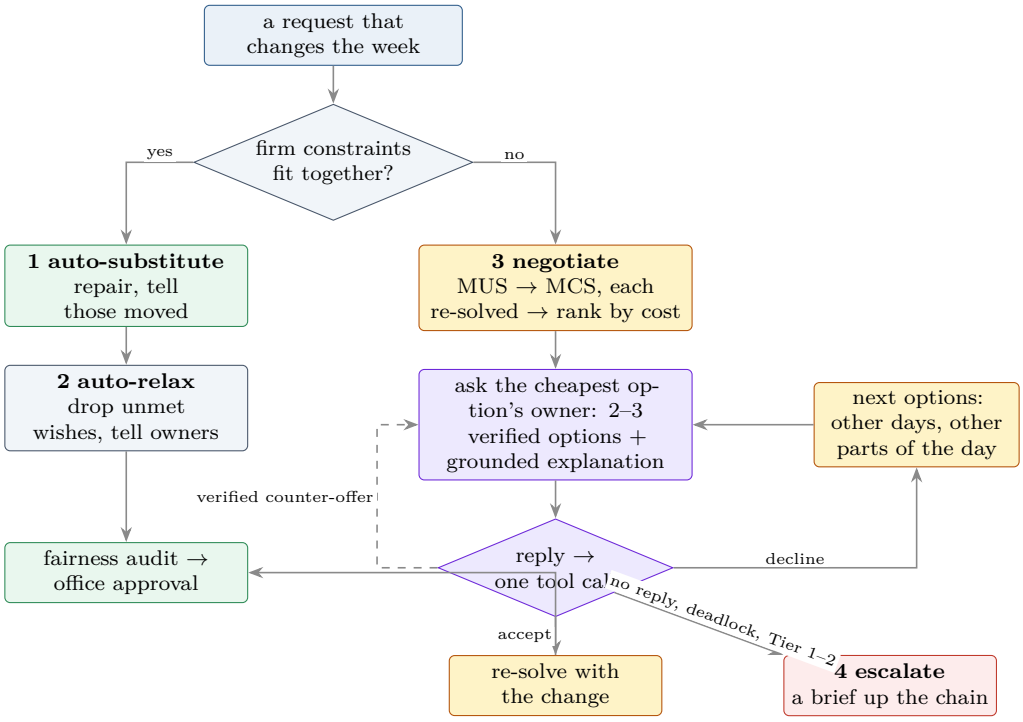


Figure 3: The resolution ladder. The solver decides what may be offered; the model only explains the offers and reads the replies.

Worked example: two practicals, one GPU lab (illustrative)

Two teachers ask for the only GPU lab on Thursday afternoon. **The MUS** is the two requests plus the lab's capacity: remove any one and the week fits. **The MCSs**, each re-solved into a full timetable, are, for example, (a) the first practical moves to Friday 14:00 in the GPU lab, (b) the second moves to Tuesday 11:00, or (c) the first stays on Thursday in a lab without GPUs. Ranked by cost (the owner's tier, scores and concession history, and how many other classes move), the cheapest option's owner is asked first, with options (a) and (b) and an explanation built from the facts ("Lab 3 is the only lab with GPUs; both practicals were requested for Thursday 14:00–17:00"), never from the other teacher's private reason. If the owner declines, the next options avoid Friday afternoon; if they accept, the week is re-solved with the change and sent to the office for approval.

Explanations are built from numbered facts (the constraints in the conflict, the rules, the verified

options, the ledger) that name constraints, never private reasons; the model writes claims citing fact IDs, and a checker drops any claim that cites nothing or mentions a day, number or name absent from its facts. **The concession ledger** credits each accepted concession by importance × magnitude, lowers the chance of asking the same person again, and decays each semester. **Approval**: every change that moves a class waits for the timetable office.

4.5 The chat: timetable questions and memory

A lookup adapter chooses *what* to look up; 16 lookup tools compute the answer from the week's timetable and the records (free hours, classes, free rooms, who is free, changes, the calendar, the asker's requests and rules, clashes, move options, counts, gaps, fairness, bookings, a what-if dry run). The model writes names as the message writes them; code resolves them, roles included ("the HOD" is whoever holds the role today), and every choice is checked, with a problem sent back to the model at most three times. Four generations were built: **v1** (one lookup, names listed in the prompt), **v2** (roles, shared courses), **v3** (up to five lookups and ten new tools, names as written; in use) and **v4**, a typed *lookup vocabulary* like the request one: a part per thing asked, each tool with only its own fields, clock times and periods instead of slots and week numbers, and request and rule parts that keep their own words. A message mixing questions, a rule question and a request is answered part by part: questions from the timetable, the rule question from the handbook, and the request sent into the pipeline.

Memory runs first: given the newest message and the four before it, the memory adapter answers `{"same"}`, the message rewritten to stand alone, or a question back, and a rewrite may name only what the conversation names. **Memory v2** fills a request from the answer in the *same* message ("when am I free on Friday? move my DS lecture there"), and asks which time when the answer has several.

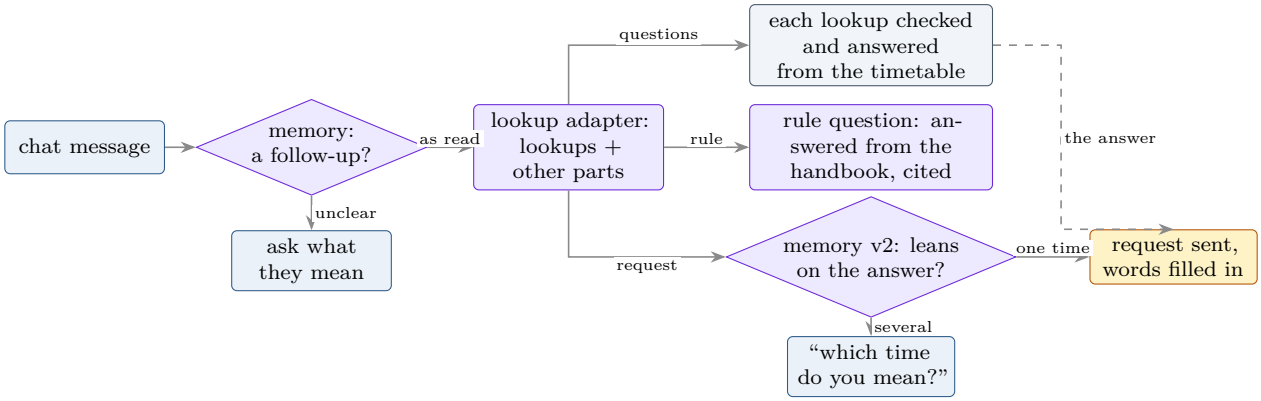


Figure 4: One chat message with questions, a rule question and a request. The model chooses what to look up; code answers from the timetable.

Worked example: the same message in lookup v3 and in the lookup vocabulary (v4)

"is Khan sir free Thu after lunch? also move my DS lecture there"
v3 (flat choices; the model counts slots and weeks, and the request is a wordless `none`):

```
{"lookups": [{"tool": "free_hours", "teachers": ["khan sir"], "days": ["Thu"],
              "hours": [5, 6, 7], "week": 15},
             {"tool": "none", "week": 15}]}
```

v4 (a part per thing asked, clock times, the request keeps its words):

```
{"parts": [{"tool": "free_hours", "who": {"teachers": ["khan sir"]},
            "when": {"days": ["Thu"], "after": "14:00"}},
           {"tool": "request", "text": "move my DS lecture there"}]}
```

In both, code finds who "Khan sir" is and answers from the timetable. With v4, memory v2 then rewrites the request part from the answer ("move my Data Structures lecture to Thursday 2–3 pm"), which v3 cannot, since its request part has no words.

4.6 The coordinator agent

As an alternative to the fixed pipeline, a tool-calling coordinator works a whole request through about 30 typed tools in five scopes: read the department, draft changes, check the draft (a repair solve of every affected week), talk to people (ask the requester, offer a waiver to the owner of a blocking rule), and end (submit, decline, escalate). Only a commit gate changes state, re-checking every change with the validator whatever the agent claimed, so every model mistake becomes an error the model sees rather than a bad timetable. Traces are labelled by the scenario's verdict or the office's approval and exported for SFT, and as DPO pairs (a refused turn against its fix; a good against a bad episode of the same request).

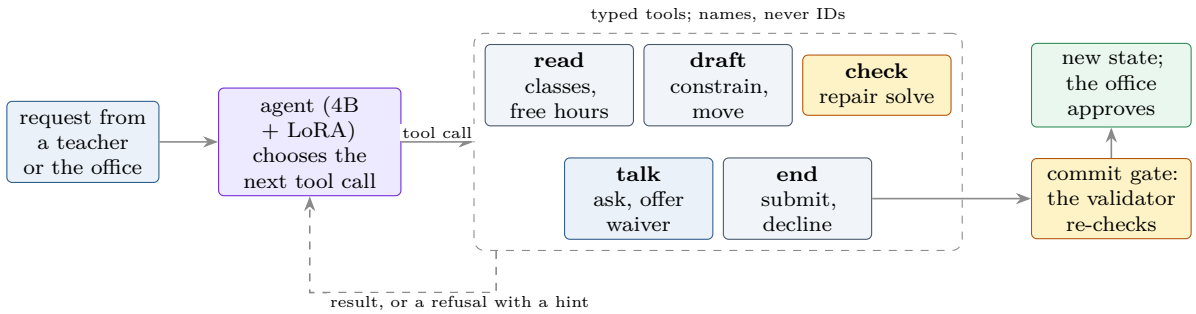


Figure 5: The coordinator agent's loop. The model never writes the timetable: only the commit gate changes state, and every refused call becomes feedback the model sees on its next turn.

4.7 Fine-tuning

One base, an adapter per task. Qwen3.5-4B (Q8_0 GGUF) is served once by `llama-server` with every adapter loaded at scale zero; each call switches on its task's adapter. Each adapter is retrained and evaluated alone, so a change to the chat never puts request reading at risk.

Which layers, and why. Every adapter uses rank 16, $\alpha = 16$, no dropout, on the linear layers named `q/k/v/o_proj` and `gate/up/down_proj` outside the vision tower. Qwen3.5 is a hybrid: of its 32 layers, 24 use Gated DeltaNet linear attention and every fourth uses full attention. The pattern therefore adapts the full attention of 8 layers (3.1M weights) and the MLP of all 32 (18.1M), about 21M weights or 0.5% of the model; the linear-attention projections, embeddings, output head and vision tower stay frozen. Full fine-tuning would need roughly four times the 17 GB fp32 model in memory, beyond two 15 GB T4s. Attention is adapted because our tasks are mostly *finding and copying* the right words of the prompt; the MLPs because the model must *transform* phrases into a vocabulary it never saw, and adapting all linear layers is the standard LoRA practice. Embeddings and output head are frozen because no new tokens are needed and their shared matrix (636M weights) would make each adapter larger than a gigabyte. Leaving out the linear-attention layers was a side effect of a name pattern written for ordinary transformers, not a choice (Section 8).

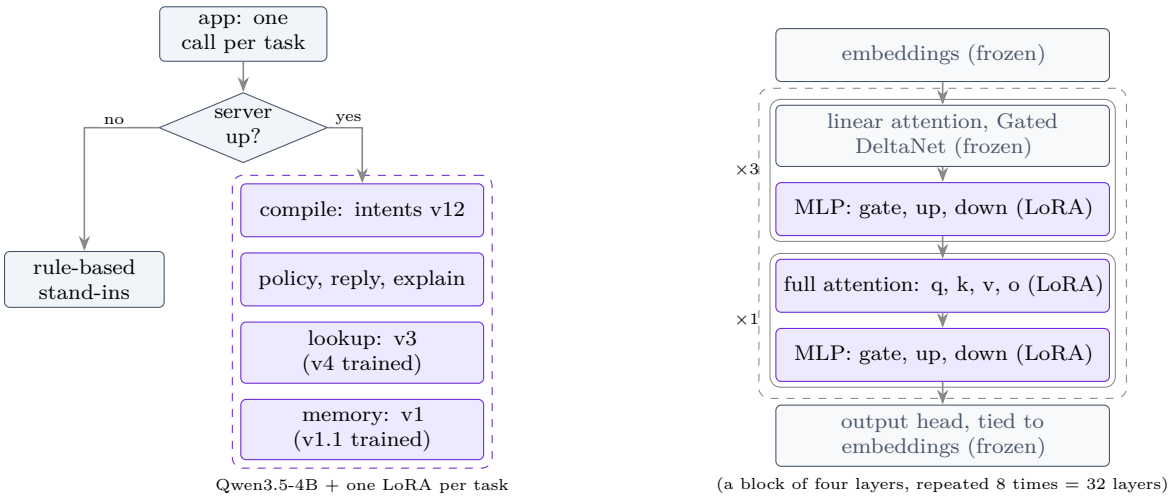


Figure 6: Left: one base model serves every task, an adapter switched on per call, with rule-based stand-ins when the server is away. Right: which layers the adapters train (violet): the MLP of all 32 layers and the attention of the 8 full-attention layers, about 21M weights (0.5% of the model).

Training. Loss is on the answer only; the prompt is masked. Learning rate 2×10^{-4} for a new adapter and 5×10^{-5} when continuing from the previous version (with part of its rows replayed), cosine schedule. The lookup and memory adapters train one epoch in fp32 (fp16 overflowed on the T4s, which lack bf16); the intents adapter trains three epochs in fp16 with the loss scale started at 2^8, saving the adapter at a third, two thirds and all of its rows (`read-033/067/100`). Adapters are exported as LoRA GGUF files of tens of MB.

Setting	**Value**	**Why**
Rank, alpha	16, 16	Narrow tasks need little capacity; $\alpha/r = 1$ keeps the learning rate's meaning fixed
Dropout, bias	0, none	One pass over varied generated rows gives little to overfit
Loss	On the answer only	The prompt is given at run time; learning it would teach one department's details
Learning rate	2×10^{-4} new; 5×10^{-5} continuing	Continue from the previous adapter without overwriting it
Schedule	Cosine, 5 warm-up steps	A gentle start, decaying to zero
Precision	fp32 (lookup, memory); fp16, loss scale 2^8 (intents)	T4s lack bf16; plain fp16 overflowed (`nan` gradients)
Max length, batch	2,048 tokens, 1 row (lookup, memory); 512 tokens, 16 rows (intents)	Lookup rows are 900–1,300 tokens with the prompt; intents rows under 300
Epochs	1 (lookup, memory); 3 (intents)	More passes memorise wordings; the third intents epoch teaches the smallest skills

Table 2: Training settings, shared by every adapter except where stated.

Training data. No real user's messages were used. Every row is generated from a known answer, or reworded by Gemini and kept only if reading it back gives the same answer, and every row passes the same gates whatever its source (Figure 7). The request reader's test split is frozen by a fingerprint, so every version is scored on the same 417 requests, and test templates and wordings never appear in training.

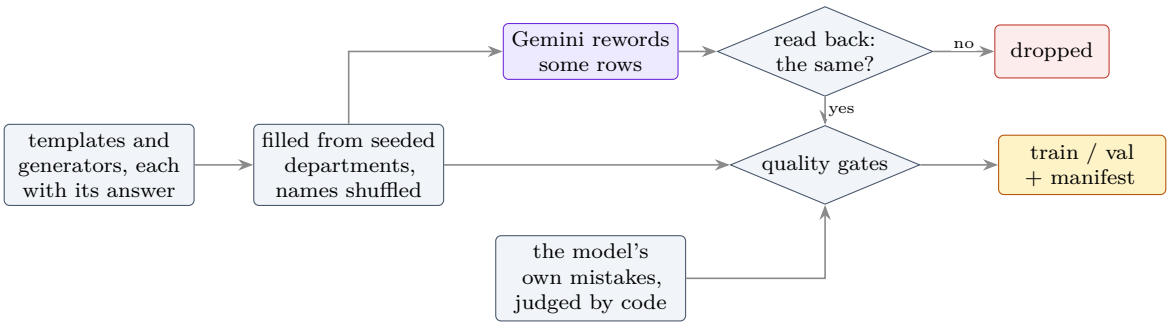


Figure 7: How training rows are made. Every source passes the same gates; the test split and test wordings are kept apart.

5 Experiment Runs

Experiment	What was run	Models
Main negotiation	Ours, B3, B4: 60 scenarios × 3 seeds (180 paired runs each); A1–A3: stratified 30, seed 0; B1: stratified 20; B2 and oracle: all 60 (no model); offline replica of ours, A1 and B2	Ours: Qwen3.5-4B + LoRA; baselines: Gemini 3.5 Flash-Lite; simulators: model-voiced
Request reading	151-request compile test; 417-request frozen intent test; 26 held-out suites; v6–v13	Rule parser, Gemini, 2B/4B LoRA, intents v6–v13 at each curve point
Policy, retrieval	Allow/deny, citations, obligations; recall@1/3/5 and MRR for three retrievers	Gemini, 4B LoRA; BM25, dense, hybrid
Replies, safety	100 replies; 48–50 safety cases through the real pipeline, also with a parser that obeys every injection	Base 4B, Gemini, 4B LoRA, regex
Timetable questions	Sets A, B2, B3, C, D (362 each); new-kind sets; 200 long messages; 26 chat cases through the app	Regex, Gemini, lookup v1–v4, v3.2
Memory	400 test conversations, memory then lookup; 64 plural conversations; 8 memory-v2 chat cases	Memory v1 (v1.1 trained)
Agent	49–50 families × 10 episodes over two departments; best-of-n; held-out rewritten requests	Base 4B, Gemini, agent v3–v7 (SFT, DPO)

Table 3: Experiment runs. Every model answer is cached by model and prompt, so any table can be re-scored without model calls and interrupted runs resume.

Compute. Training ran on free Kaggle sessions (two T4 GPUs, 12-hour limit; runs resume from checkpoints): a lookup or memory adapter takes 3–4 hours, an intents version one session. Serving and evaluation used one shared GPU machine running `llama-server`; the solver runs on CPU. Gemini was used for baselines, simulators, rewordings and the judge, within free-tier quotas.

5.1 Configurations compared in the main experiment

Table 4 lists every configuration of the main experiment, and the question each one answers.

Configuration	What it does	Question it answers
Ours	MCS options, each re-solved; tiers and ledger; grounded explanations; replies read as typed tool calls	The system under test
B1 LLM only	The model writes the timetable directly; no solver	Can a model just do it?
B2 Solver only	Same solver, tiers and objective, but the cheapest repair is imposed without asking anyone	Is negotiation needed, or only a solver?
B3 Unverified	As ours, but the model invents the options and they are offered unchecked	How often are invented options infeasible?
B4 Filtered	As B3, but CP-SAT drops infeasible inventions and the model tries again (up to 3 calls a round)	The strongest "model invents" system (primary baseline)
Oracle	CP-SAT with every stakeholder's hidden flexibility	The best possible outcome (upper bound)
A1 / A2 / A3	Ours with flat weights / free-text explanations / a regex reply reader	What each part contributes (Section 7)

Table 4: Configurations of the main experiment.

5.2 Metrics

- **Correct outcome**: the run ends as the oracle's does (an agreement, or an escalation where none is possible). A scenario is *resolved* when every affected stakeholder accepted an offered alternative and the combined repair re-solves feasibly.
- **Agreement**: the run ends in an agreement rather than an escalation. **Rounds**: rounds of messages before it ends.
- **Oracle gap**: how much worse the final timetable is than the oracle's, by the oracle's own objective (lower is better).
- **Invalid options**: offered options that are infeasible; for B4, how many inventions the solver filtered.
- **Fairness**: the Gini coefficient of the weighted concession burden over a "semester" of scenarios run in order with one ledger, and the largest individual burden.
- **Explanations**: private-reason leaks, and claim faithfulness (the share of claims traceable to a fact).
- **Reading**: *outcome match*, whether a reading leads to the same change to the timetable as the correct one (a different wording that does the same thing counts as right), and *silently wrong*, a reading that changes the timetable wrongly without asking. Timetable questions are scored by what each lookup resolves to after the code resolves names and weeks, not by its wording.

Protocol. Ours, B3 and B4 run the same 60 scenarios with the same three seeds, so runs are paired, and correct outcomes are compared with McNemar's test. Each scenario carries one stakeholder style, balanced within each kind of conflict, and replies are random within the style, so no system can succeed by exploiting a fixed reply rule. Scenarios run in order with one concession ledger per configuration and seed, so the burden builds up as over a semester among the same people.

6 Evaluations

6.1 Main experiment: solver-grounded negotiation

A run is *correct* when its final status matches the oracle's (the solver with every stakeholder's hidden flexibility). The baselines use the much larger Gemini 3.5 Flash-Lite, so they are not handicapped on language.

System	n	Correct outcome	Agreement	Rounds	Oracle gap	Invalid options
B1 LLM only	20	0.0%	0.0%	–	–	80% of timetables
B2 Solver only	60	31.7%	0.0%	–	0.131	–
B3 Unverified	180	83.3%	66.7%	1.47	0.443	51.0%
B4 Filtered	180	75.6%	51.1%	1.48	0.628	5.7 filtered/sc.
Ours	180	**89.4%**	**78.9%**	**1.38**	**0.041**	**0**

Table 5: Main results. McNemar's test on correct outcome: ours vs B4 $p = 4.6 \times 10^{-6}$, vs B3 $p = 0.013$; differences in rounds are not significant.

A model writing the timetable (B1) breaks it every time. A solver alone (B2) finds a valid repair but imposes it, and is right only a third of the time: hidden flexibility matters, so negotiation is needed. Half of the options a model invents are infeasible (B3). Filtering them (B4) is *worse* than not filtering: the solver rejects most inventions, the survivors are fewer and worse, and fewer people agree. Ours offers only feasible options and lands almost on the oracle (gap 0.04). **Failures**: 19 of 180 runs, all safe escalations where the oracle found agreement: a stakeholder did not reply in time (11), or the acceptable option was not among those offered (8). No run offered an infeasible timetable.

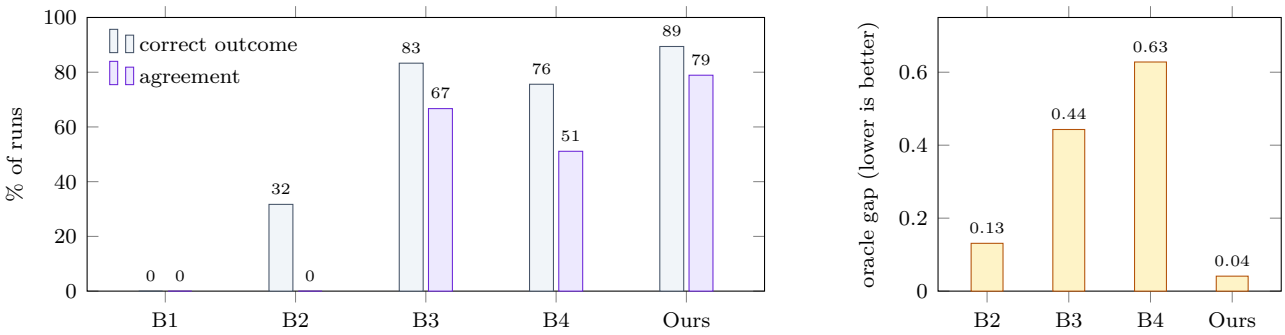


Figure 8: Main experiment. Left: correct outcome and agreement. Right: distance from the oracle's timetable (B2 imposes a cheap repair, so its gap is small, but it is right only a third of the time).

6.2 What the main results show

Three things stand out. First, **verification after the fact is not enough**: filtering invented options (B4) removes the infeasible ones but leaves fewer and worse offers, so people agree less often than when the unchecked inventions are offered (B3), and both fall well short of offering verified repairs from the start. Second, **the solver alone is not enough either**: imposing the cheapest repair (B2) ignores what people would have accepted, and is right in under a third of scenarios; asking is what finds the hidden flexibility. Third, **the gain does not come from a bigger model**: ours runs on a 4B model against a much larger one for the baselines; the advantage comes from what the model is allowed to offer. Rounds are statistically equal across systems, so H1 and H2 hold for outcome and quality but not for speed.

Hyp.	Evidence	Verdict
H1	51% of B3's options infeasible; B4 needs 3.97 generation calls per scenario and still agrees least; rounds not significantly higher	**supported** (rounds n.s.)
H2	Correct 89.4% vs 75.6%; oracle gap 0.04 vs 0.63; rounds equal within the confidence interval	**supported** (except rounds)
H3	Grounded vs free text (A2): 0 vs 12 private-reason leaks; claim faithfulness 0.98, measured automatically	**partly**
H4	Ledger vs flat weights: Gini 0.241 vs 0.224 (model runs), 0.199 vs 0.224 (offline); one conflict per scenario	**not yet**

Table 6: Hypotheses.

6.3 Component evaluations

Reading requests. On 151 test requests both fine-tuned models beat the prompted Gemini on compiling (97.8% exact against 92.4%, atom F1 0.996 against 0.977). Scored by *outcome match* (does the reading change the timetable as the correct one would) on the frozen 417-request split, the rule parser reaches 51.6% (21.1% silently wrong), intents v11 95.4% (2.4%), and v12 98.1% (1.9%) after its second epoch; the adapter served, after its third epoch, reaches 96.4%. On compositional requests (limits, relations, choices) v12 reaches 85.7% with none silently wrong.

Policy. The fine-tuned adapter reaches 99% allow/deny accuracy (Gemini 98%), deny recall with a cited rule of 0.90 (0.80), and recall of the make-up class obligation of 1.00 (0.21). Hybrid retrieval reaches recall@5 of 0.93 on corpus requests with the reading's hints.

Replies and safety. On 100 stakeholder replies the fine-tuned adapter reaches 100% tool and argument accuracy (prompted base 65% / 100%; Gemini 100% / 85.5%), with no unsafe action by any reader. Every case of the safety set is handled safely: no unapproved publish, no authority violation, no injection success, no private leak.

Timetable questions and memory.

Test	Regex	Gemini	v1	v2	v3	v3.2	v4
A: app's department (362)	242	320	329	**347**	346	339	337
C: names shuffled (362)			329	343	**346***	334	340
D: unseen, shared courses (360)			327	**345**	343*	335	333
Long and mixed messages, whole message (200)					111	119	**140**
question parts (561)					440	445	**483**
requests found (123)					112	117	**122**
Chat cases through the app (26)					25	25	**26**
Memory then lookup, end to end (400)					387	386	**388**

Table 7: Timetable questions. *recounted with the current labels (at least); v1 and v2 were scored with the labels of their time. The adapters score alike across departments: they read rather than memorise.

Coordinator agent. On the 19-family default suite the SFT agent v4 passes 100% (prompted base 66.7%, Gemini 85.1%), and 100% of held-out and stress families. On the full 50-family suite over two departments, agent v6 (SFT then DPO) passes 412/480 (85.8%), v7 SFT 387/490 (79.0%) and v7

DPO 381/490 (77.8%).

7 Ablations

An ablation takes the system and removes or replaces exactly one part, or compares two versions that differ in one way only, so that any change in the results can be put down to that part. This section explains each ablation in plain terms: what was changed, why that change isolates the part, what happened, and what it means. The five tables summarise the numbers; the text gives the reasoning.

7.1 Negotiation: which part of the method matters?

The main experiment already contains the two biggest ablations: **B2** removes negotiation (the solver imposes its cheapest repair) and **B3/B4** remove grounding (the model invents the options). Three further ablations keep everything else of our system and change one part each (Table 8).

A1: flat weights instead of tiers and the ledger. Every constraint gets the same score and the concession ledger is ignored, so the negotiator no longer prefers to ask people who have given way less often. This is the test of H4. On the model runs, how often conflicts were settled did not change (90.0%), and the burden was *not* spread more evenly with the ledger (Gini 0.241 with it, 0.224 without). On the full offline replica (all 60 scenarios, template explanations, scripted replies) the ledger did lower the Gini (0.199 against 0.224) and was right once more often (88.3% against 86.7%), but one scenario in 60 is far from significant ($p = 1.0$). The reason is in the benchmark, not the ledger: each scenario has a single conflict, so the ledger rarely has a history to act on. The ledger's effect can only show over a sequence of requests among the same people.

A2: free-text explanations instead of grounded ones. The model writes its explanation freely from the raw requests, instead of from numbered facts that name constraints but never private reasons. Settlement is unchanged, because people's decisions in the simulators do not depend on the wording, but the free-text explanations **leaked 12 private reasons** (a colleague's medical appointment, for example) where grounded ones leaked none, and 98% of grounded claims trace back to a fact. Grounding is what keeps private reasons private (H3).

A3: a keyword reader instead of tool calls. Each reply is turned into an action by keyword rules instead of the fine-tuned model's typed tool call. In the scenarios settlement is again unchanged, because the simulated replies are short and regular. On the 100-reply benchmark, written the way people reply, the keyword reader picks the right action only **66%** of the time against 100% for the model: it gets every counter-offer and every vague reply wrong, reading "could we do Friday instead?" as a counter to an offered option rather than a proposal of a new time, and "which room would that be in?" as a refusal (Figure 10).

B2 and B3/B4 seen as ablations. Without negotiation the outcome is right in 31.7% of scenarios, and the largest burden on one person rises from 5.25 to 9.19: the cheapest repair keeps landing on the same people. Without grounding, half the invented options are infeasible (B3), and filtering them costs 3.97 model calls per scenario and still settles fewer conflicts (B4).

Ablation	What changes	Result
A1 flat weights	No tiers or ledger; every constraint scores the same	Settlement unchanged (90.0%); Gini 0.224 vs 0.241 (model runs); offline 86.7% vs 88.3% correct, Gini 0.224 vs 0.199 ($p = 1.0$)
A2 free-text explanations	The model explains freely instead of from solver facts	Settlement unchanged; **12 private-reason leaks** vs 0
A3 regex replies	A keyword reader turns replies into actions instead of the model's tool calls	Settlement unchanged on the simulators; on 100 real-style replies **66%** tool accuracy vs 100%
B2 (no negotiation)	The cheapest repair imposed without asking	31.7% correct; largest individual burden 9.19 vs 5.25
B3 / B4 (no grounding)	The model invents the options, unchecked / filtered	83.3% / 75.6% correct; 51% infeasible / 3.97 calls per scenario

Table 8: Negotiation ablations. The parts do not change how often conflicts are settled on these scenarios; they change what stakeholders see, how replies are read, and how the burden falls.

7.2 Reading requests

These ablations ask what makes the request reader good: the model, its size, the vocabulary, the data, or the training schedule (Table 9).

Fine-tuned against prompted, and model size. A 4B model with a LoRA adapter compiles 97.8% of the 151 test requests exactly, against 92.4% for the much larger Gemini given the same instructions: a small model trained on the task beats a large one that is only told about it. The 2B and 4B adapters compile equally well, but the 4B one decides more often what to do with a message (action accuracy 99.3% against 96.5%), so the 4B base was chosen.

The model against rules. The rule parser, which is the system's stand-in when the model server is away, reaches 51.6% outcome match and is *silently wrong* on 21.1% of requests: it changes the timetable wrongly without asking. Intents v12 reaches 98.1% with 1.9% silently wrong. This gap is why requests that would change the timetable wait for the model instead of being read by the rules.

Names in the prompt or resolved by code. Up to schema 3 the prompt carried the department's whole directory, and the model wrote IDs. From schema 4 the model writes names as the message gives them, and code finds the IDs. The median prompt fell from 1,800 to 300 characters and an epoch of training data from 3.9M to 0.57M tokens, making training about seven times cheaper, and the first reader trained this way (v6) scored 1.00 on the frozen test. Taking the lookup out of the model cost nothing and saved most of the compute.

Targeted generators, and their cost. Each version adds generators for exactly what the previous one got wrong. v12 added four for v11's misses and lifted relations from 45.0% to 77.5% and the test split to 98.1%. v13 added two more and fixed what they aimed at (answered questions 0.68 → 1.00, same-day parts 0.28 → 0.73), but lost compound requests (1.00 → 0.65) and "out for me" messages (0.98 → 0.53), as Figure 9 shows. New rows dilute old skills: each generator adds about 2.5% of the data, so each new skill is learned at the expense of others unless the old ones are rebalanced.

Epochs. The intents adapter is saved after each of its three epochs. The second-epoch adapter is better on the common requests (test 0.98 against 0.96), while the third is what teaches the newest, smallest skills (relation pairs 0.07 → 1.00). Validation loss cannot tell them apart (it is near zero at every epoch), so the choice has to be made on the held-out suites.

Updating against training from the base. v7 was trained as an update of v6 on 335 new rows and lost skills v6 had (moves 0.82 → 0.70). From v8 on, every version is trained from the base on the whole recipe.

Ablation	Comparison	Result
Fine-tuned vs prompted	4B LoRA vs Gemini 3.5 Flash-Lite	97.8% vs 92.4% exact compile
Model size	Qwen3.5-2B vs 4B, same data	Same compile (97.8%); action accuracy 96.5% vs **99.3%**
Model vs rules	Rule parser vs intents v12	51.6% vs 98.1% outcome match; silently wrong 21.1% vs 1.9%
Names in the prompt	IDs and a directory in the prompt vs names as written, resolved by code	Prompt 1,800 → 300 characters; tokens per epoch 3.9M → 0.57M; v6 scored 1.00 on the frozen test
Targeted generators	v11 vs v12 (+4 generators)	Test 95.4% → 98.1%; relations 45.0% → 77.5%; compositional 78.6% → 85.7%
More generators	v12 vs v13 (+2 generators)	Test 96.4% → 98.3%; gains and losses by suite in Figure 9
Epochs	v12 after epoch 2 vs 3	Test 0.98 vs 0.96; relation pairs 0.07 → 1.00 only in epoch 3
Update vs from base	v7 updated from v6 vs trained from the base	The update lost moves (0.82 → 0.70)

Table 9: Request-reading ablations (outcome match on the 417-request frozen split unless stated).

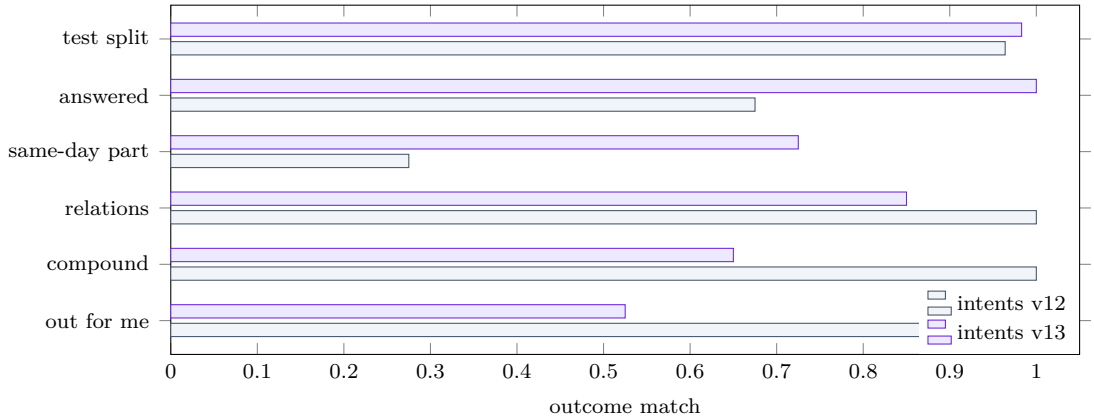


Figure 9: Adding generators is a trade: v13 learns what its two new generators teach (answered, same-day part) and the test split improves, but older skills fall.

7.3 Policy, replies and safety

Retrievers. Three retrievers find the rules for the policy check: word matching (BM25), dense embeddings, and the two fused (Figure 10). Each wins somewhere. On requests with the hints the reading adds (a one-off absence adds "make-up"), the hybrid is best (93.4% of correct rules in the top 5). On questions worded unlike the rules, dense search is best (87.8%), because it matches meaning, not words. On raw requests without hints, BM25 is best (77.4%) and dense search collapses (45.3%): "I'm at a conference in week 7" never mentions make-up classes, so only the hint can bring that rule in. The hybrid is never far from the best, so the app uses it.

The policy model. Prompted Gemini and the fine-tuned adapter allow or deny almost equally well (98% and 99%), but only the adapter learned that a teacher's absence obliges a make-up class: it names that obligation every time it applies (recall 1.00), where Gemini almost never does (0.21). A rule that is always true in the data is easy to learn and easy for an instruction-following model to forget.

Reply readers. Four readers turn the same 100 replies into tool calls (Figure 10). The prompted base model often picks the wrong action (65%) but fills its arguments correctly; Gemini picks the right action but gets arguments wrong (85.5%); keywords fail on counter-offers and vague replies (66%); the fine-tuned adapter is right on both counts every time. No reader ever took an unsafe action, because every call is checked.

Safety by architecture. To test whether safety depends on the model, the safety set was run with a parser that obeys every injected instruction. It still handled 49 of 50 cases safely, with no injection succeeding, no change made without authority and no private reason revealed: the guarantees come from the pipeline (instructions removed before reading, authority checked by rule, every change

re-solved and approved), not from the model behaving well.

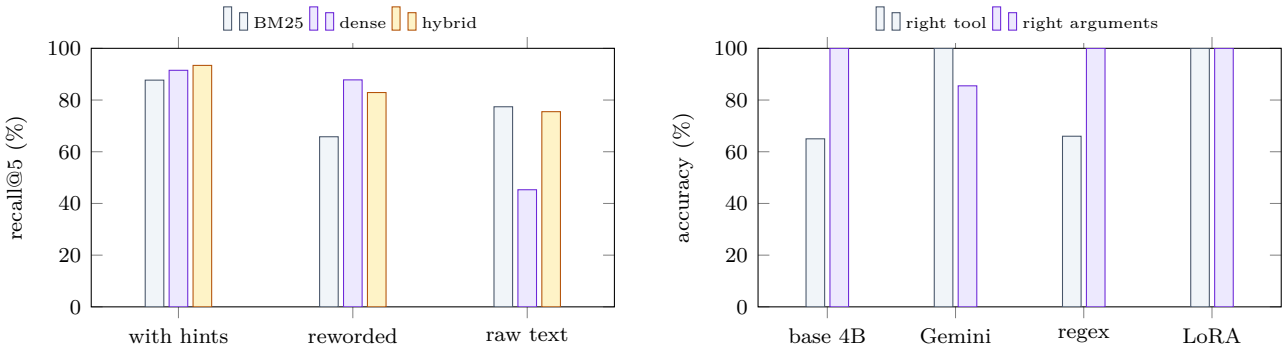


Figure 10: Left: rule retrieval by query type; each retriever wins somewhere, the hybrid is never far behind. Right: reading 100 stakeholder replies; only the fine-tuned adapter gets both the action and its arguments right.

Ablation	Comparison	Result
Retriever	BM25 vs dense vs hybrid, recall@5	With hints: 87.7 / 91.5 / **93.4**%; reworded: 65.8 / **87.8** / 82.9%; raw text: **77.4** / 45.3 / 75.5%
Policy model	Prompted Gemini vs fine-tuned 4B	Make-up obligation recall 0.21 vs **1.00**; cited deny recall 0.80 vs **0.90**
Reply reader	Prompted base vs Gemini vs regex vs LoRA	Tool accuracy 65 / 100 / 66 / **100**%; arguments 100 / 85.5 / 100 / **100**%
Safety by architecture	Our parser vs a parser that obeys every injection	Still 49/50 safe, **0** injection successes

Table 10: Policy, reply and safety ablations.

7.4 Timetable questions and memory

A model against a regex. The regex reader, which the chat falls back to, answers 242 of the 362 single questions right; prompted Gemini 320; the first fine-tuned lookup adapter 329. Reading what a question asks is mostly a language problem, and the trained small model handles it best.

Reading or memorising? The same 362 questions were asked in the app's department, in that department with every teacher's name, the head of department and the course titles shuffled, and in a department the models never saw, with shared and co-taught courses. Scores barely move (v2: 347, 343, 345; v3: 346, 346, 343), so the adapters read the question and let code find the names, rather than remembering who is who in one department.

Output format: the cleanest ablation. v3.2 and v4 were trained on *exactly the same rows*, continuing from the same adapter with the same settings; only the answer's format differs: v3.2 writes v3's flat choices (any field on any tool, slot numbers, week numbers), v4 writes the lookup vocabulary (a part per thing asked, each tool with only its own fields, clock times and periods, request parts with their words). Any difference between them is the format's. On single questions they are alike (339 and 337 of 362). On long and mixed messages v4 is clearly better (Figure 11): 140 whole messages right against 119, 483 of 561 question parts against 445, and 122 of 123 requests found against 117. In the chat it fills "there" from the answer in 7 of 8 cases against 5. A typed vocabulary helps most exactly where messages are long and mixed, because each part is written on its own, with only the fields its tool has.

New data and forgetting. Against v3, v4 lost 25 single questions and gained 16. The losses are the same few kinds in every department: "in week N" read as week $N-1$, "can I have Thursday free?" read as a question, "why was X's class cancelled?" looked up instead of kept private. None of them appears in v4's new rows: continuing from an adapter on new data forgets what the new data does not mention, unless enough of the old rows are replayed.

Memory. The 400 test follow-ups were looked up as typed, and after memory rewrote them: 146 right without memory, 387–388 with it. Without memory, fewer than four follow-ups in ten work at all. **Memory v2** fills a request from the answer in the same message; it depends on the lookup format,

since a request part must keep its words to be filled. **Plurals**: memory v1 already handles "these days" (26/26) and "those times" (17/17), but replaces a person with "the same time for X" only 9 times in 21, which is what v1.1's new rows teach.

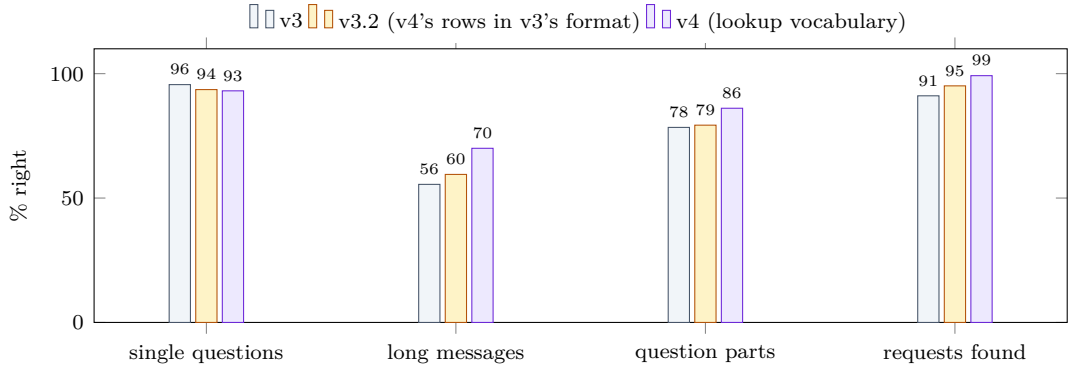


Figure 11: The output-format ablation. v3.2 and v4 differ only in format; the typed vocabulary gains most on long and mixed messages.

Ablation	Comparison	Result
Model vs regex	Regex vs Gemini vs lookup v1 (set A)	242 vs 320 vs 329 of 362
Memorisation	The same questions in the app's, a shuffled and an unseen department	v2: 347 / 343 / 345; v3: 346 / 346 / 343
Output format	v3.2 vs v4: *the same training rows*, flat choices vs the lookup vocabulary	Long messages 119 vs **140**; requests found 117 vs **122**; single questions alike (339 vs 337)
New data, forgetting	v3 vs v4 on single questions	v4 lost 25 and gained 16 on set A
Memory	Without vs with memory (400 follow-ups)	146 vs **387–388** looked up right
Memory v2	v3.2 vs v4 request parts	5/8 vs **7/8** chat cases
Plurals	Memory v1 on three kinds of plural	26/26, 17/17, 9/21

Table 11: Timetable-question and memory ablations.

7.5 Coordinator agent

Fine-tuning. On the 19-family suite the prompted 4B base passes 66.7%, Gemini 85.1% and the fine-tuned agent 100% (Figure 12). The agent learns from its own verified trajectories: which tool to call, in what order, and when to stop.

SFT against SFT followed by DPO. DPO trains on pairs of a better and a worse episode of the same request. On the full 49-family suite v7 scores 79.0% after SFT and 77.8% after DPO, a difference within the noise of two runs of the same job (up to 10 episodes in one family). Its held-out preference accuracy was only 0.65 on 48 pairs: with a few hundred pairs from one source, DPO barely moved the model.

Epochs. Validation loss shows that a second epoch helps a little and a third never does; more episodes help more than more passes.

Code fixes against more training. Five families failed for reasons in the tools, not the model (a daily cap on teaching days, for example). Fixing the tools took those families from 24 to 47 of 50 without retraining; the exam-week family went from 0 to 9–10 of 10 once the agent could read the examination regulations. When a model keeps failing at something, the cause is often in its tools or inputs, and code is the cheaper fix.

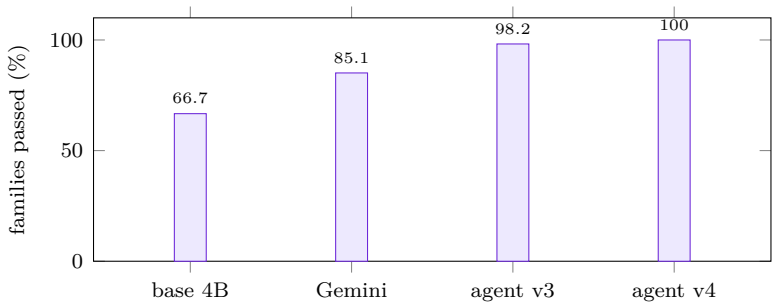


Figure 12: The coordinator agent on the 19-family default suite: fine-tuning a 4B model on verified trajectories beats a much larger prompted model.

Ablation	Comparison	Result
Fine-tuning	Base 4B vs Gemini vs SFT (19 families)	66.7% vs 85.1% vs **100%**
SFT vs SFT + DPO	v7 SFT vs v7 DPO (49 families × 10)	79.0% vs 77.8%: within noise
Epochs	Epoch 1 vs 2 vs 3	The second helps little, the third never
Code fixes vs training	v7 SFT with and without five tool fixes	Those five families 24/50 → **47/50**
Policies	With vs without the exam regulations	Exam-week family 0/10 → 9–10/10

Table 12: Agent ablations. Two runs of the same job differ by up to 10 episodes in one family: differences under that are noise.

7.6 What the ablations show overall

Across all five areas the same pattern appears. **Where code can decide, code should**: verified offers beat invented ones, names resolved by code made training seven times cheaper at no cost, the safety guarantees held even with a parser that obeys every injection, and tool fixes did what more training had not. **Where language must be read, a small model trained on the task beats a large one told about it**, in reading requests, policy obligations, replies, timetable questions and agent behaviour. **What remains is mostly about data**: new rows dilute old skills, continued training forgets what new data omits, and the ledger cannot show its effect on single-conflict scenarios. Each of these points to a change in data or benchmark, not in the architecture.

8 Limitations

- **H4 is not supported.** With one conflict per scenario the ledger has little to act on; scenarios with a sequence of requests among the same people are needed.
- **Mixed models in the main comparison.** Ours runs on the fine-tuned 4B model, the baselines on the larger Gemini; a run with the same model on both sides would separate the method from the model.
- **Human validation.** Explanation faithfulness and the judge are measured automatically, not yet against human ratings, and the faculty pilot waits for ethics clearance.
- **Synthetic data and simulated people.** Every dataset is generated or team-written, the handbook is a stand-in, and negotiation partners are simulators with four styles; real people may behave otherwise.
- **Training-data gaps.** No training row asks about a *range* of times ("between 10 and 2") or dates, so every lookup adapter reads only the ends; typed date-range requests ("cancel all my classes from 9 to 13 Nov") are asked back, since the request vocabulary holds weeks, not dates (the click-based range builder handles them). Continuing an adapter forgets what its new rows do not mention unless enough old rows are replayed (v4 and "in week N"), and adding generators can cost older skills (v13's compound requests).

- **Linear attention not adapted.** The adapters' layer pattern misses Qwen3.5's 24 linear-attention layers; adapting them is untested.
- **Infrastructure.** One shared GPU server; when it is unreachable the rule-based stand-ins are much weaker (51.6% against 96.4% on reading requests).

9 Conclusions

Letting the solver define what may be offered, and the language model only explain it and read the answers, settles infeasible timetable requests more often and far closer to the best possible outcome than letting a model invent options, even with the same solver filtering them: 89.4% against 75.6%, an oracle gap of 0.04 against 0.63, and not one infeasible offer. The same division of labour carries through the system: small fine-tuned adapters read language into typed vocabularies and beat a much larger prompted model on every reading task, while code resolves names, dates and rules and answers every question from the timetable. The ablations locate what remains: the ledger needs longer histories to show its effect, grounded explanations are what keep private reasons private, and most reading errors trace to gaps in generated training data rather than to the architecture, which points to the next step: training data drawn from real requests.

Reproducibility

Every dataset is generated from fixed seeds, with its answers known by construction. The whole system runs offline on its deterministic stand-ins, and its tests need no model or API key. Every model answer is cached by model and prompt, so every table can be scored again without calling a model, and interrupted runs resume. The adapters are trained on free Kaggle GPUs from the same data, and served with `llama-server` on one base model.

References

[1] E. K. Burke, S. Petrovic. Recent research directions in automated timetabling. *European Journal of Operational Research*, 140(2):266–280, 2002.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[3] S. Kambhampati et al. Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks. *ICML*, 2024.

[4] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[5] M. H. Liffiton, K. A. Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008.

[6] L. Perron, F. Didier. CP-SAT. Google OR-Tools, https://developers.google.com/optimization.